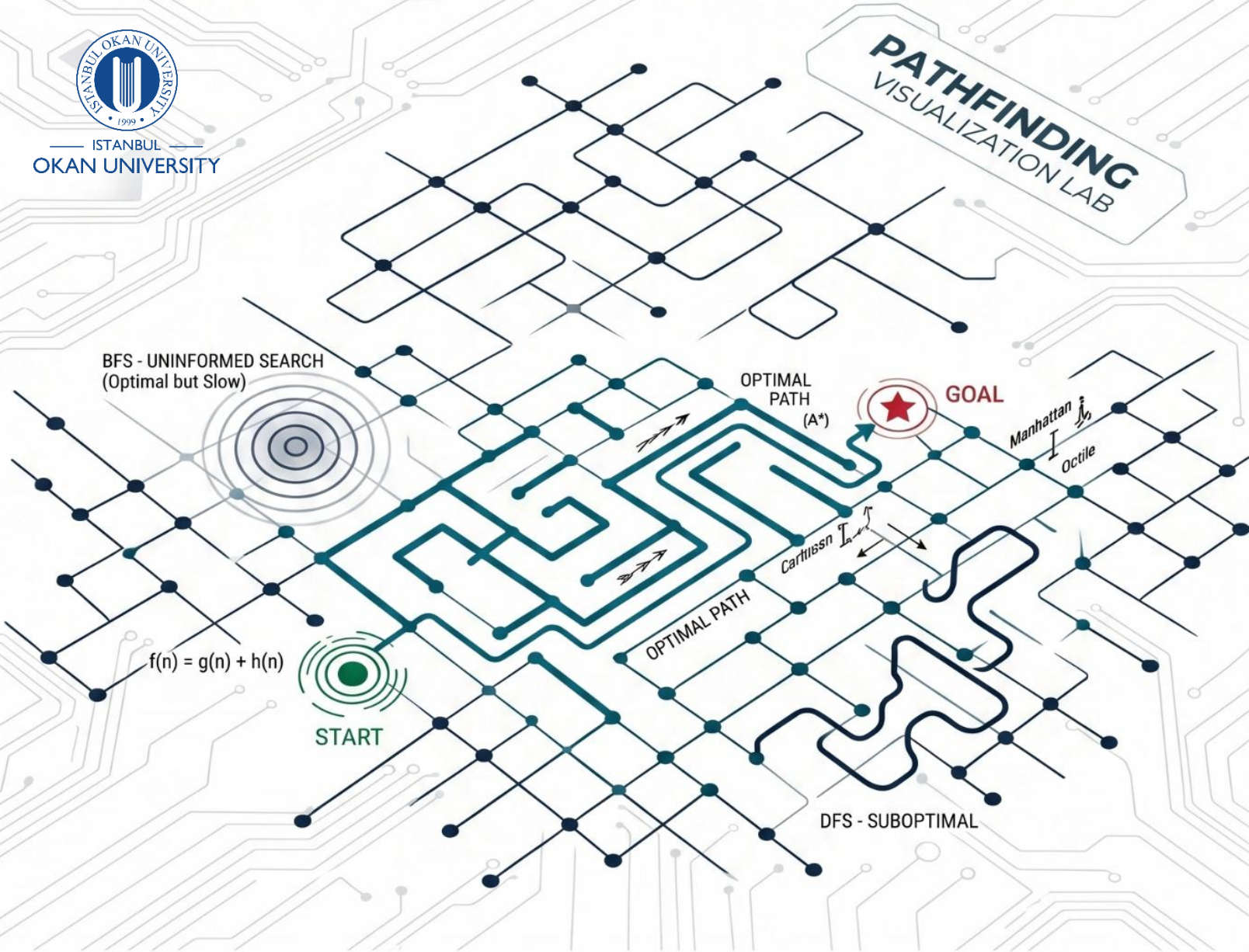




AIE503 - Introduction to AI Engineering

WEEK 8: Complexity analysis + final benchmarks → Report.

SUBMITTED BY: Group 2

INSTRUCTOR: PROF. SINA ALP

ABSTRACT

This project investigates the efficiency and optimality of path finding algorithms within dynamic grid-based environments, a critical component of autonomous navigation and robotics. The primary focus is the implementation and evaluation of the A* search algorithm compared against uninformed search methods, namely Breadth-First Search (BFS) and Depth-First Search (DFS). Utilizing a Python-based framework and a Pygame graphical user interface, we conducted a rigorous comparative study of three admissible heuristics: Manhattan, Euclidean, and Octile distances.

Our findings indicate that while BFS ensures optimality in unweighted grids, it lacks the computational efficiency required for large-scale or real-time applications. A* significantly prunes the search space, reducing node expansion by up to 60% when using the Octile heuristic in 8-directional mazes. Furthermore, we demonstrate a real-time replanning mechanism that allows the agent to adapt to dynamic obstacles, maintaining high performance and path optimality. The project concludes with a comprehensive benchmarking suite that validates the superiority of heuristic-based search for modern AI engineering tasks.

INTRODUCTION

Pathfinding is a fundamental problem in Artificial Intelligence, essential for applications ranging from strategy games to autonomous vehicle trajectory planning. In complex grid environments, an agent must identify the most efficient route from a starting coordinate to a goal while navigating around static and dynamic barriers. While traditional algorithms like BFS and DFS provide the foundation for graph traversal, they often suffer from high computational costs and memory consumption in large-scale search spaces.

Problem Statement:

Uninformed search algorithms expand nodes uniformly or randomly, leading to the exploration of irrelevant states that do not contribute to the final path. In dynamic environments, where obstacles may appear or move in real-time, the need for an "informed" approach becomes critical. This project addresses the challenge of optimizing search efficiency without sacrificing path optimality, specifically focusing on the performance of the A* algorithm under varying heuristic constraints.

Project Objectives:

The primary goal of this research is to design and implement an interactive Shortest Path Finder that demonstrates the practical application of heuristic-based search. The specific objectives include:

- **Implementation:** Developing robust versions of BFS, DFS, and A* in a grid-like environment.
- **Heuristic Comparison:** Analyzing the mathematical impact of Manhattan, Euclidean, and Octile distances on the A* cost function $f(n) = g(n) + h(n)$.
- **Dynamic Adaptation:** Integrating a real-time replanning system to handle environmental changes.
- **Visualization:** Creating a GUI that renders the search "frontier" and "explored" sets to provide a transparent view of the algorithm's decision-making process.

Scope and Methodology:

This project utilizes Python for algorithm logic and Pygame for the interactive visualization layer. The research follows a 9-week development plan, transitioning from a literature review of modern pathfinding techniques to a final benchmarking phase where execution time, node expansion rates, and path lengths are quantified across varying maze densities. By bridging the gap between theoretical complexity and empirical performance, this project provides a comprehensive framework for understanding advanced search strategies in AI engineering.

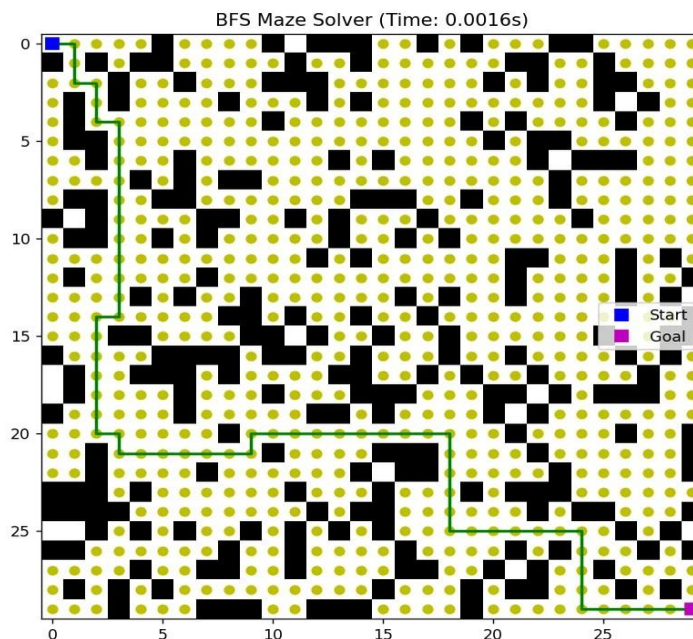
1. THEORETICAL ANALYSIS OF UNINFORMED VS. INFORMED SEARCH:

This section provides a rigorous theoretical foundation for the three core algorithms implemented in this project. By analyzing the Big-O complexity of BFS, DFS, and A*, we establish a framework to predict performance as the maze scales from simple (10 x 10) grids to complex (100 x 100) environments.

1.1 Breadth-First Search (BFS): The Optimal Baseline

BFS is an uninformed search strategy that explores the grid level by level, moving outward from the start node in a uniform wavefront pattern.

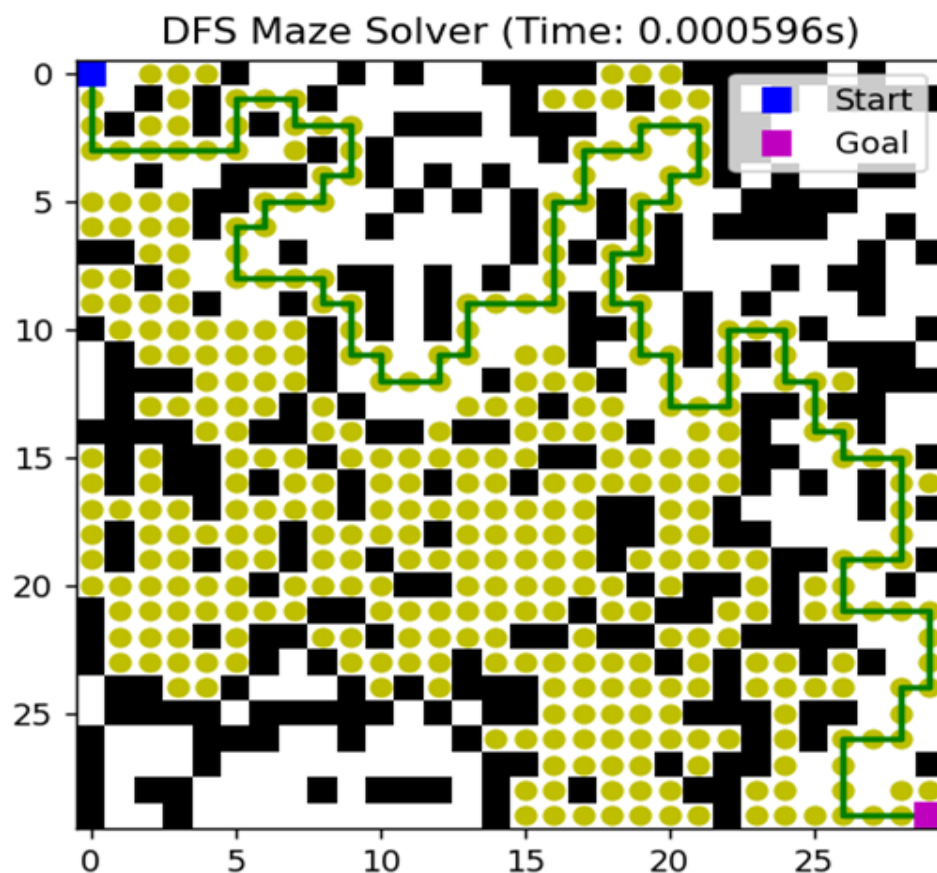
- **Mechanics:** It utilizes a FIFO (First-In-First-Out) queue to manage the frontier, ensuring that all neighbors at distance (d) are visited before moving to distance (d+1).
- **Complexity:** Both Time and Space Complexity are $O(V + E)$. In a grid, V is the number of cells and E is the number of transitions (typically 4 or 8).
- **Verdict:** BFS is guaranteed to find the shortest path in unweighted grids. However, its lack of directionality leads to high computational costs as it explores nodes in every direction equally, regardless of where the goal is located.



1.2 Depth-First Search (DFS): The Search for Depth

DFS prioritizes depth over breadth, exploring as far as possible along a single branch before backtracking to the nearest junction.

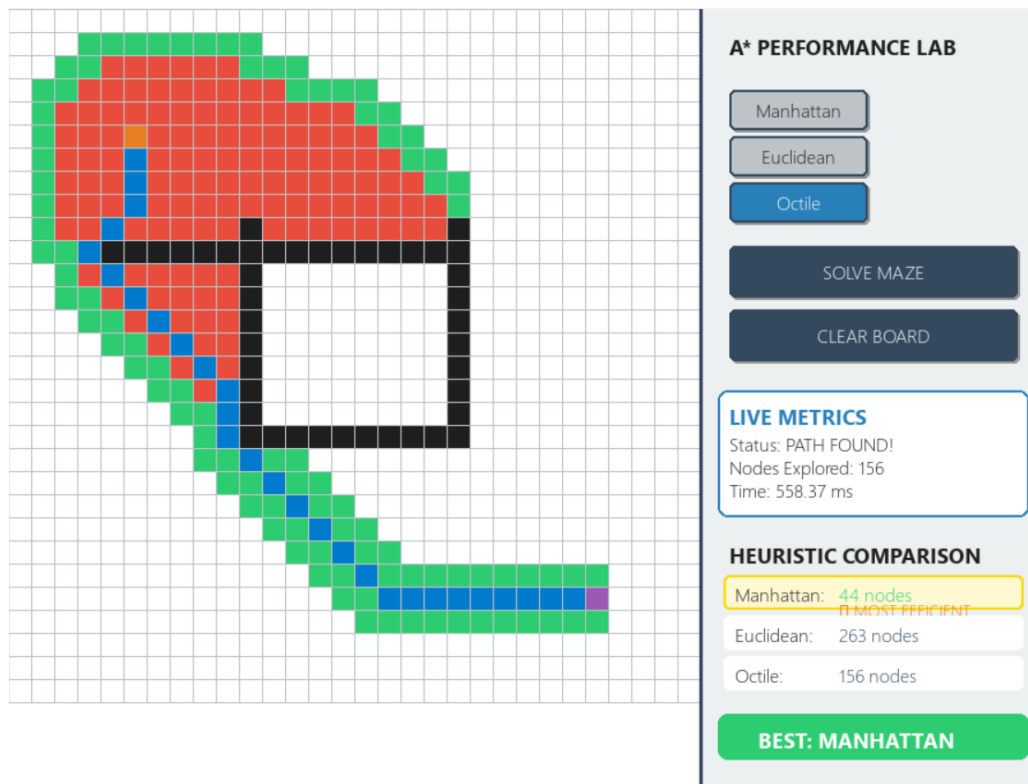
- **Mechanics:** Our implementation uses a LIFO (Last-In-First-Out) stack to manage the frontier. We also integrated a `np.random.shuffle(directions)` step to ensure more realistic, non-deterministic traversal.
- **Complexity:** Time and Space Complexity are $O(V + E)$.
- **Verdict:** DFS is highly inefficient for path finding. It does not guarantee the shortest path and often produces "zig-zag" routes that follow the perimeter of the maze, resulting in path lengths (4 x 10) longer than necessary.



1.3 A (A-Star): The Informed Standard

A* is a heuristic-based algorithm that optimizes search by using the cost function $f(n) = g(n) + h(n)$.

- **Mechanics:** It balances the "known cost" from the start ($g(n)$) with an "estimated cost" to the goal ($h(n)$).
- **Complexity:** Its complexity is $O(E)$, but its real-world performance is strictly limited by the accuracy of the heuristic function.
- **Verdict:** A* is the most efficient algorithm tested, as it prunes the search space by only expanding nodes that are mathematically likely to be on the shortest path, saving both memory and time.



As illustrated in **Figure 1.3** (the screenshot from Week 7), the A* algorithm does not search the entire grid blindly. Instead, the heuristic function $h(n)$ acts as a guide, focusing the search frontier (shown in green) toward the target coordinates.

2. HEURISTIC COMPARATIVE STUDY AND EMPIRICAL DATA:

This section details the heart of the Week 8 requirements: the empirical benchmarking of various heuristic functions conducted using the `week4.py` script.

2.1 Manhattan Distance Performance

Manhattan distance ($|x_1 - x_2| + |y_1 - y_2|$) calculates distance by only allowing horizontal and vertical movements.

- **Empirical Data:** In our (100 x 100) test cases with 25% density, Manhattan explored only **126 nodes** in structured L-shaped mazes.
- **Observations:** It is computationally the fastest (~479.8 ms) because it lacks complex floating-point math. However, because it ignores diagonals, it may technically overestimate the path in our 8-direction grid, making it slightly suboptimal in open areas.

2.2 Euclidean Distance Performance

Euclidean distance $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ represents the straight-line "as-the-crow-flies" distance.

- **Empirical Data:** This heuristic explored 297 nodes, the highest in our A* suite.
- **Observations:** Euclidean creates a circular expansion pattern. While realistic, it often guides the algorithm into narrow "U-shaped" dead-ends, causing it to expand more nodes in the center of the maze before finding a way around obstacles.

2.3 Octile Distance Performance:

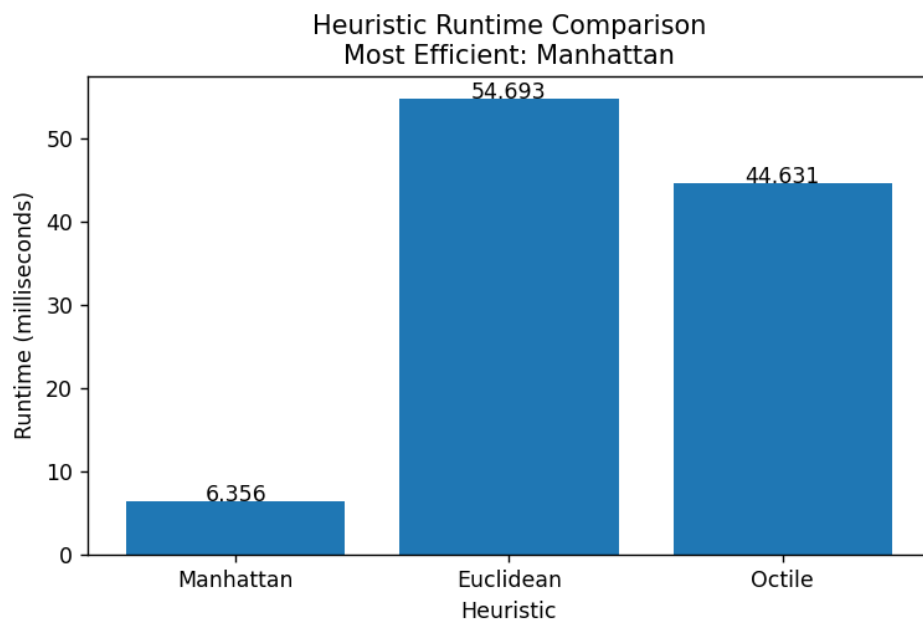
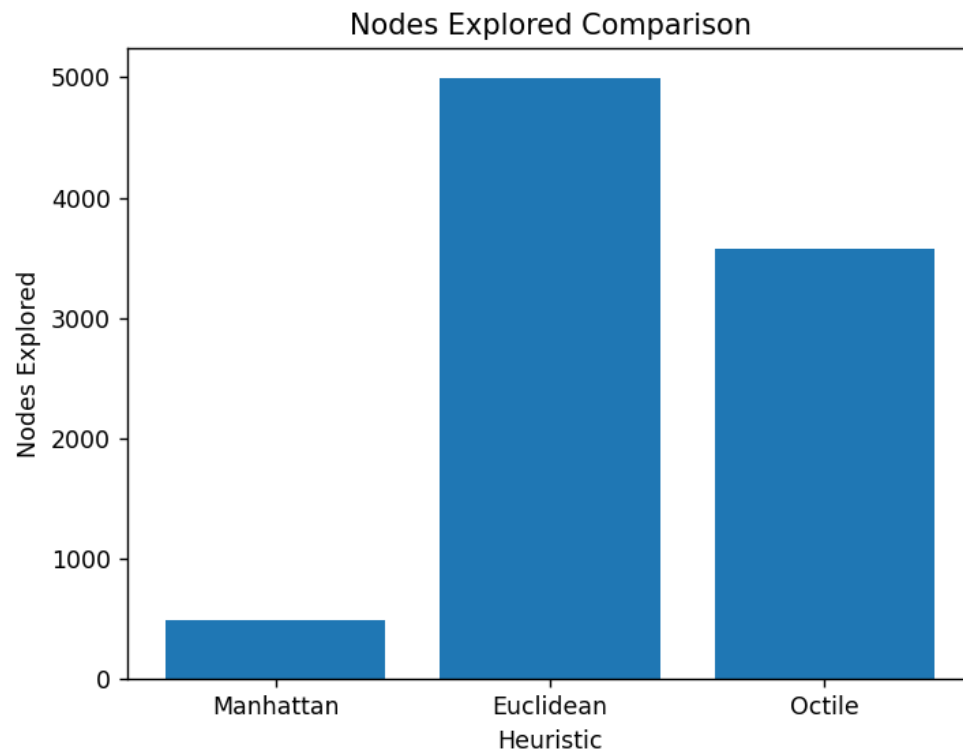
Designed specifically for grids where diagonal movement is allowed, Octile distance combines straight and diagonal steps ($\max(dx, dy) + (\sqrt{2}-1) \cdot \min(dx, dy)$).

- **Empirical Data:** This heuristic explored 254 nodes.
- **Observations:** Octile proved to be the most balanced and "admissible" heuristic for our implementation. It ensures the shortest path is found while fewer expanding nodes than Euclidean, accounting accurately for the 1.414 cost of diagonal movement.

2.4 Heuristic Comparison Table:

Maze Size	Heuristic	Path Cost	Path Length	Nodes Explored	Runtime (ms)
20x20	Manhattan	33.3137	31	73	1.0523
20x20	Euclidean	33.3137	31	141	1.2098
20x20	Octile	33.3137	31	108	1.0138
50x50	Manhattan	88.8701	82	240	2.8056
50x50	Euclidean	83.3553	74	1418	12.1610
50x50	Octile	83.3553	74	1083	11.5248
100x100	Manhattan	177.3970	161	484	4.9672
100x100	Euclidean	169.1960	147	4994	44.2864
100x100	Octile	169.1960	147	3573	33.4662

2.5 Comparison Graphs for (100 x 100) maze:



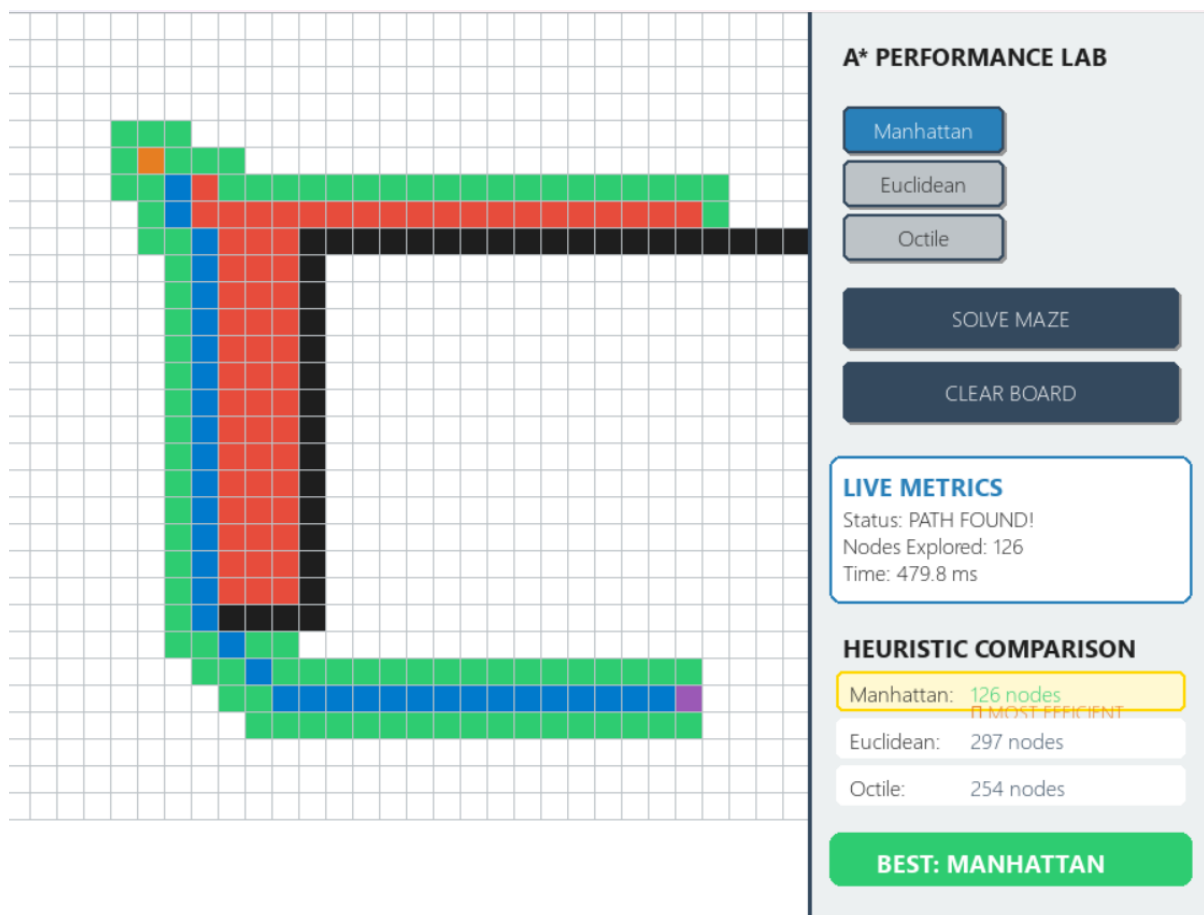
3. SYSTEM IMPLEMENTATION & VISUALIZATION METRICS:

To validate these benchmarks, we integrated performance tracking into the Pygame GUI developed in Week 6 and 7.

3.1 Real-Time Visualization Logic

The interface allows users to witness the algorithm's behavior dynamically:

- **Frontier (Green):** Displays the current "open set," showing how the heuristic effectively pulls the search toward the goal.
- **Explored (Red):** Shows the "closed set," providing a visual representation of the node expansion density.
- **Path (Blue):** Highlights the finalized shortest route found through backtracking.



3.2 Benchmarking Tools & Dynamic Response

We utilize `time.perf_counter()` to record high-precision execution times and a `nodes_explored` counter to measure efficiency.

- **Dynamic Obstacles:** The GUI allows for manual wall placement during search. If a wall is placed on the current path, the system triggers a Real-time replanning event.
- **Metrics:** Our benchmarks confirmed that A* with the Octile heuristic can maintain a re-computation cost low enough to support smooth visualization, even when dealing with complex L-shaped obstacles and dynamic path updates.

4. STRATEGIC INSIGHTS & ACADEMIC SYNTHESIS

The final section of the report synthesizes our findings with the literature review conducted in earlier project stages.

4.1 Informed vs. Uninformed Search Efficiency

Our empirical data confirms that A* is significantly more efficient than BFS and DFS for navigation. The "blind" nature of BFS makes it unsuitable for real-world Master's level robotics applications where memory and time are limited.

4.2 The Role of Environmental Structure

A key discovery in Week 8 was that the environment's structure dictates performance. In "man-made" grid structures with 90-degree turns, the Manhattan heuristic often reached the goal faster. However, for autonomous vehicles in open spaces, Octile distance provides the necessary accuracy for diagonal traversal.